

ndelement and ndmesh: numerical discretisation in Rust

Timo Betcke ¹ and **Matthew W. Scroggs** ²

¹ Department of Mathematics, University College London ² Advanced Research Computing Centre, University College London

DOI: [10.xxxxxx/draft](#)

Software

- [Review](#) ↗
- [Repository](#) ↗
- [Archive](#) ↗

Editor: [Prashant Jha](#)

Reviewers:

- @alexfikl
- @joeybernard

Submitted: 16 April 2026

Published: unpublished

License

Authors of papers retain copyright
and release the work under a
Creative Commons Attribution 4.0
International License ([CC BY 4.0](#)).

Summary

When working with partial differential equations (PDEs) that arise from problems in science or engineering, it is common to use numerical methods to obtain approximate solutions. Many numerical methods for PDEs, such as the finite element method (Ciarlet, 1978; Johnson, 2009) and boundary element method (Steinbach, 2008), involve the discretisation of a domain into a mesh of polygonal or polyhedral cells, and the definition of a set of (usually) polynomial basis functions on each cell. A library implementing one of these methods will typically be built around two core pieces of functionality: the creation and storage of a mesh, and the definition of basis functions on a cell.

The Rust programming language (Klabnik et al., 2026) has received significant attention in recent years, and the community using it for scientific applications is steadily growing. Rust is a compiled language, with strict typing and memory safety enforced by the compiler, supported by high quality modern tooling and package management. There are a number of features of Rust that make it a highly powerful language for writing scientific applications.

Using Rust's concept of a trait, a developer can define abstract interfaces. A trait can then be implemented for one or many structures, with the Rust compiler ensuring at compile time that all necessary traits are implemented. By allowing many traits to all be separately implemented for the same structure, Rust allows for the implementation of complex behaviours without any need for complicated hierarchical class inheritance. Traits also make Rust libraries naturally extensible, as if another developer implements your trait on their structure, then their structure can in many cases be seamlessly used with the functionality of your library.

Rust is supported by modern and powerful tooling via the cargo build system and package manager, with code linting, code quality and testing frameworks included by default. Releases of Rust crates are hosted online at crates.io, and these crates can be easily added as dependencies of your project by adding them to your crate's Cargo.toml configuration file.

In order to enable the implementation of various discretisation methods using Rust, we have developed the libraries `ndelement` and `ndmesh`, with both libraries developed as sub-crates within the `nd` (numerical discretisation) repository (Betcke & Scroggs, {2024–2026}). The libraries provide core functionality to define basis functions and handle meshes. These libraries make use of the Rust linear algebra library `rlst` (Betcke et al., {2023–2026}) and the parallelisation library `rsmpi` (Brown et al., {2015–2026}).

Software design

In this section, we outline the key features of the `ndelement` and `ndmesh` libraries.

39 ndelement

40 The ndelement library includes traits for a finite element and a reference-mapped finite element,
41 as defined in definitions 2 and 3 of Scroggs et al. (2026). By providing these traits, we allow
42 potential users to build on top of ndelement by implementing these traits for their new element
43 implementations. Using some pseudo-code for simplicity and with documentation hidden, the
44 trait for a finite element looks like this:

```
pub trait FiniteElement {
    type T;
    type CellType;

    fn cell_type(&self) -> Self::CellType;

    fn dim(&self) -> usize;

    fn value_shape(&self) -> &[usize];

    fn value_size(&self) -> usize;

    fn tabulate(
        &self,
        points: &Array2D<Self::T>,
        nderivs: usize,
        data: &mut Array4D<Self::T>,
    );

    fn tabulate_array_shape(
        &self,
        nderivs: usize,
        npoints: usize
    ) -> [usize; 4];

    fn entity_dofs(
        &self,
        entity_dim: usize,
        entity_number: usize,
    ) -> Option<&[usize]>;

    fn entity_closure_dofs(
        &self,
        entity_dim: usize,
        entity_number: usize,
    ) -> Option<&[usize]>;
}
```

45 This trait defines the most general finite element, with no unnecessary assumptions made. It
46 includes functions to:

- 47 ■ see what cell type the finite element is defined on (cell_type). Importantly, the return
48 type of the function cell_type is a defined by the implementation of the trait, allowing
49 developers of other libraries to return the correct cell type even when it is not included
50 in the ReferenceCellType enum that we use for the elements we have implemented,
51 allowing us to avoid any assumption that elements can only be defined on certain cell
52 types.
- 53 ■ get the number of basis functions of the finite element (dim);
- 54 ■ obtain the shape of the (possibly vector-, matrix- or tensor-valued) basis functions
55 (value_shape and value_size);
- 56 ■ evaluate the basis functions at a set of points (tabulate and the utility function
57 tabulate_array_shape); and
- 58 ■ find out which basis functions are associated with each sub-entity of the cell (entity_dofs
59 and entity_closure_dofs).

60 The trait for a reference-mapped finite element looks like this:

```
pub trait MappedFiniteElement: FiniteElement {
    type TransformationType;

    fn lagrange_superdegree(&self) -> usize;

    fn push_forward(
        &self,
        reference_values: &Array4D<Self::T>,
        nderivs: usize,
        jacobians: &Array3D<Self::T>,
        jacobian_determinants: &[Self::T],
        inverse_jacobians: &Array3D<Self::T>,
        physical_values: &mut Array4D<Self::T>,
    );

    fn pull_back(
        &self,
        physical_values: &Array4D<Self::T>,
        nderivs: usize,
        jacobians: &Array3D<Self::T>,
        jacobian_determinants: &[Self::T],
        inverse_jacobians: &Array3D<Self::T>,
        reference_values: &mut Array4D<Self::T>,
    );

    fn physical_value_shape(&self, gdim: usize) -> Vec<usize>;

    fn physical_value_size(&self, gdim: usize) -> usize;

    fn dof_transformation(
        &self,
        entity: Self::CellType,
        transformation: Self::TransformationType,
    ) -> Option<&DofTransformation>;

    fn apply_dof_permutations(
        &self,
        data: &mut [Self::T],
        cell_orientation: i32,
    );

    fn apply_dof_transformations(
        &self,
        data: &mut [Self::T],
        cell_orientation: i32,
    );

    fn apply_dof_permutations_and_transformations(
        &self,
        data: &mut [Self::T],
        cell_orientation: i32,
    );
}
```

61 In addition to the functions that make up a finite element, reference-mapped finite elements
62 additionally include functions to:

- 63 ■ get the maximum Lagrange degree of the finite element's basis functions (as defined in
- 64 section 2.4 of Scroggs et al. (2026)) (lagrange_superdegree);
- 65 ■ map function values between the reference cell and a physical cell (push_forward and
- 66 pull_back);
- 67 ■ obtain the shape of the basis functions after the mapping to a physical cell
- 68 (physical_value_shape and physical_value_size); and
- 69 ■ apply the degree-of-freedom (DOF) permutation and transformation method introduced
- 70 in Scroggs, Dokken, et al. (2022) and Scroggs & Wells (2026) (dof_transformation,
- 71 apply_dof_*).

ndelement includes implementations of arbitrary degree Lagrange, Raviart–Thomas (Raviart & Thomas, 1977), and Nédélec (Nédélec, 1980) finite elements, with the above traits implemented for these elements. The correctness of these implementations is verified using DefElement’s element verification feature (Scroggs et al., 2026); details of this verification are available at defelement.org/verification/ndelement.html. The currently available elements in ndelement are summarised in the following table:

Cell type	Supported element types
Interval	Lagrange
Triangle	Lagrange, Raviart–Thomas, Nédélec (first kind)
Quadrilateral	Lagrange, Raviart–Thomas, Nédélec (first kind)
Tetrahedron	Lagrange, Raviart–Thomas, Nédélec (first kind)
Hexahedron	Lagrange, Raviart–Thomas, Nédélec (first kind)

ndmesh

The aim of the ndmesh library is to handle meshes of polygonal or polyhedral cells. ndmesh can load and save meshes from/to gmsh format files (Geuzaine & Remacle, 2009) and create uniform meshes of some built-in shapes such as unit squares and cubes. If the library is compiled with the mpi feature activated, meshes can be distributed over multiple MPI processes, using either Scotch (Pellegrini & Roman, 1996) or Coupe (Chevalier et al., 2024) to partition the cells, or by providing a custom partition.

ndmesh includes traits that define a minimal user interface to a mesh. The trait for a mesh includes functions to iterate through sub-entities. The entities given by this iterator must implement the entity trait that includes functions to obtain structures implementing the topology and geometry traits that can be used to obtain information about the connectivity between entities and the physical position of the entity (respectively).

A key strength of this library is that implementing these traits for any implementation of a mesh (either within ndmesh or defined in an external library) can allow users to (for example) use functions to assemble finite element matrices built on top of ndmesh with any of these mesh implementations. Within ndmesh, implementations of a *single element mesh* (where every cell has the same cell type is represented using the same degree finite element) and a *mixed mesh* (where cells can be a mixture of cell types and represented using a mixture of polynomial degrees) are available.

The meshes in ndmesh are implemented with a strict separation between the topology and geometry of the mesh. This approach is heavily inspired by the representation of meshes in DOLFINx (Baratta et al., 2023).

State of the field

A number of libraries that provide similar functionality to ndelement already exist. These include the Python library FIAT (Kirby, 2004) that is used as part of the Firedrake framework (Ham et al., 2023) and the C++/Python library Basix (Scroggs, Baratta, et al., 2022) that is one of the components of FEniCSx (Baratta et al., 2023). The Python library Symfem (Scroggs, 2021) allows users to find the basis functions of finite elements symbolically, which is useful for debugging and prototyping but too slow to be used for real applications.

There are a large number of open source libraries available that implement the finite element method—including, but not limited to, deal.II (Arndt et al., 2025) FEniCSx (Baratta et al., 2023), Ferrite.jl (Carlsson et al., {2015–2026}), Firedrake (Ham et al., 2023), MFEM (Andrej et al., 2024), and ngsolve (Schöberl, 2014). Each of these libraries necessarily either includes

the implementation of finite elements and a finite element mesh, or uses an implementation of these from another library.

A few implementations of finite elements in Rust are available. The majority of these implementations are for a limited set of problems, such as FEM_2D (Corrado et al., 2023), which only looks to solve problems in two spatial dimensions. The most general purpose Rust FEM library that we are aware of is Fenris (Longva & Löschner, {2021--2023}). However, the project's readme file states that Fenris is currently "not recommended for general usage" and it appears that development ceased in 2023.

We do not know of any implementation of boundary element methods in Rust except for our own experimental implementation built on top of older versions of ndelement and ndmesh (Betcke et al., {2022--2024}).

Statement of need

While a number of finite element method libraries exist that include implementations of meshes and finite element basis functions, there is currently no well-maintained general-purpose library in Rust. We believe that the Rust ecosystem presents an excellent opportunity for very powerful finite and boundary element libraries, and that the development of solid foundations to build numerical discretisation Rust libraries on top of is very important.

Research impact

ndelement and ndmesh are the first two components that we have developed in our work towards developing a new Rust-based version of the Bempp boundary element method library. Previous versions of Bempp were implemented in C++ with a Python interface (Šmigaj et al., 2015) and in pure Python with just-in-time compiled OpenCL assembly kernels (Betcke & Scroggs, 2021). These previous versions have been used in a wide range of research applications (see e.g. Bosy et al. (2024), Chen et al. (2024), van 't Wout et al. (2015), Watson et al. (2024)). The ultimate goal of this development is that a new Rust library with a Python user interface that closely resembles Bempp-cl. This will make it easy for the userbase of Bempp to move to the new version and take advantage of the wide range of new features due to ndelement and ndmesh, such as the availability of quadrilateral and curved cells in meshes, MPI-distributed meshes, and a wider range of finite elements.

New research is already being enabled by ndelement and ndmesh. This includes work on improvements to the fast multiple method for accelerating matrix multiplication (Kailasa, 2025), the randomized strong recursive skeletonisation matrix algorithm (Fierro Piccardo, 2024), and an investigation into function spaces for operator preconditioning on quadrilateral cells (Scroggs & Fierro-Piccardo, {2025--2026}).

By developing our libraries in a modular way, we aim to allow a variety of discretisation methods to be developed on top of a set of shared libraries, removing the need for any other groups looking to develop numerical discretisation methods in Rust to first implement all of the core fundamentals included in ndelement and ndmesh. We have started developing the crate ndfunctionspace, which uses ndelement and ndgrid to create function spaces of basis functions on a mesh by assigning global degree-of-freedom (DOF) numbers to each cell. These function spaces can be used for both finite and boundary element methods, and so this development is our next step towards implementing these methods in Rust.

AI usage disclosure

AI inline suggestions from Github Copilot were used for code documentation and as general coding support tool. No agentic tools were used in the development of the libraries. All AI

suggestions were thoroughly checked before being merged into the main branch.

References

- Andrej, J., Atallah, N., Bäcker, J.-P., Camier, J.-S., Copeland, D., Dobrev, V., Dudouit, Y., Duswald, T., Keith, B., Kim, D., Kolev, T., Lazarov, B., Mittal, K., Pazner, W., Petrides, S., Shiraiwa, S., Stowell, M., & Tomov, V. (2024). High-performance finite elements with MFEM. *The International Journal of High Performance Computing Applications*, 38(5), 447–467. <https://doi.org/10.1177/10943420241261981>
- Arndt, D., Bangerth, W., Bergbauer, M., Blais, B., Fehling, M., Gassmöller, R., Heister, T., Heltai, L., Kronbichler, M., Maier, M., Munch, P., Scheuerman, S., Turcksin, B., Uzunbajakau, S., Wells, D., & Wichrowski, M. (2025). The deal.II library, version 9.7. *Journal of Numerical Mathematics*, 33(4), 403–415. <https://doi.org/10.1515/jnma-2025-0115>
- Baratta, I. A., Dean, J. P., Dokken, J. S., Habera, M., Hale, J. S., Richardson, C. N., Rognes, M. E., Scroggs, M. W., Sime, N., & Wells, G. N. (2023). *DOLFINx: The next generation FEniCS problem solving environment*. <https://doi.org/10.5281/zenodo.10447666>
- Betcke, T., Fierro Piccardo, I., Kailasa, S., & Scroggs, M. W. ({2022--2024}). *bempp-rs: Boundary element methods in Rust*. <https://github.com/bempp/bempp-rs>.
- Betcke, T., Fierro Piccardo, I., Kailasa, S., & Scroggs, M. W. ({2023--2026}). *rlst: The Rust linear solver toolbox*. <https://codeberg.org/rlst/rlst>.
- Betcke, T., & Scroggs, M. W. ({2024--2026}). *nd: Numerical discretisation in rust*. <https://codeberg.org/nd-project/nd>.
- Betcke, T., & Scroggs, M. W. (2021). Bempp-cl: A fast Python based just-in-time compiling boundary element library. *Journal of Open Source Software*, 6(59), 2879. <https://doi.org/10.21105/joss.02879>
- Bosy, M., Scroggs, M. W., Betcke, T., Burman, E., & Cooper, C. D. (2024). Coupling finite and boundary element methods to solve the Poisson–Boltzmann equation for electrostatics in molecular solvation. *Journal of Computational Chemistry*, 45(11), 787–797. <https://doi.org/10.1002/jcc.27262>
- Brown, J., Gaspar, A., Ruffwind, P., & Steinbusch, B. ({2015--2026}). *rsmpi: MPI bindings for Rust*. <https://github.com/rsmpi/rsmpi>.
- Carlsson, K., Ekre, F., & Ferrite.jl contributors. ({2015--2026}). *Ferrite.jl*. <https://github.com/Ferrite-FEM/Ferrite.jl>
- Chen, J., Jin, B., & Liu, H. (2024). Solving inverse obstacle scattering problem with latent surface representations. *Inverse Problems*, 40(6), 065013. <https://doi.org/10.1088/1361-6420/ad466a>
- Chevalier, C., Hirtz, H., Ledoux, F., & Morais, S. (2024). Coupe: A mesh partitioning platform. In E. Ruiz-Gironés, R. Sevilla, & D. Moxey (Eds.), *SIAM international meshing roundtable 2023* (pp. 43–63). Springer Nature Switzerland. https://doi.org/10.1007/978-3-031-40594-5_3
- Ciarlet, P. G. (1978). *The finite element method for elliptic problems*. North-Holland.
- Corrado, J., Harmon, J. J., Ilic, M. M., & Notaroš, B. M. (2023). FEM_2D: A rust package for 2D finite element method computations with extensive support for hp-refinement. *Journal of Open Source Software*, 8(84), 4172. <https://doi.org/10.21105/joss.04172>
- Fierro Piccardo, I. (2024). *Rusting RSRs*. talk at Scientific Computing in Rust 2024. <https://scientificcomputing.rs/2024/talks/fierro-piccardo.html>

- 201 Geuzaine, C., & Remacle, J.-F. (2009). Gmsh: A three-dimensional finite element mesh
202 generator with built-in pre- and post-processing facilities. *International Journal for Numerical*
203 *Methods in Engineering*, 79(11), 1309–1331. <https://doi.org/10.1002/nme.2579>
- 204 Ham, D. A., Kelly, P. H. J., Mitchell, L., Cotter, C. J., Kirby, R. C., Sagiya, K., Bouziani,
205 N., Vorderwuelbecke, S., Gregory, T. J., Betteridge, J., Shapero, D. R., Nixon-Hill, R.
206 W., Ward, C. J., Farrell, P. E., Brubeck, P. D., Marsden, I., Gibson, T. H., Homolya,
207 M., Sun, T., ... Markall, G. R. (2023). *Firedrake user manual* (First edition). Imperial
208 College London; University of Oxford; Baylor University; University of Washington. <https://doi.org/10.25561/104839>
- 210 Johnson, C. (2009). *Numerical solution of partial differential equations by the finite element*
211 *method*. Dover Publications.
- 212 Kailasa, S. (2025). *Modern resesarch software for fast multipole methods* [PhD thesis].
213 University College London.
- 214 Kirby, R. C. (2004). Algorithm 839: FIAT, a new paradigm for computing finite element
215 basis functions. *ACM Transactions on Mathematical Software*, 30(4), 502–516. <https://doi.org/10.1145/1039813.1039820>
- 217 Klabnik, S., Nicols, C., Krycho, C., & the Rust community. (2026). *The Rust programming*
218 *language* (third edition). No Starch Press.
- 219 Longva, A. B., & Löschner, F. ({2021--2023}). *fenris: A library for advanced finite element*
220 *computations in Rust*. <https://github.com/InteractiveComputerGraphics/fenris>.
- 221 Nédélec, J.-C. (1980). Mixed finite elements in $\mathbb{R}^3$. *Numerische Mathematik*, 35(3), 315–341.
222 <https://doi.org/10.1007/BF01396415>
- 223 Pellegrini, F., & Roman, J. (1996). Scotch: A software package for static mapping by dual
224 recursive bipartitioning of process and architecture graphs. In L. Heather, A. Colbrook, B.
225 Hertzberger, & P. Slood (Eds.), *High-performance computing and networking: International*
226 *conference and exhibition HPCN EUROPE 1996 brussels, belgium, april 15–19, 1996*
227 *proceedings* (pp. 493–498). Springer Berlin Heidelberg. [https://doi.org/10.1007/3-540-](https://doi.org/10.1007/3-540-61142-8_588)
228 [61142-8_588](https://doi.org/10.1007/3-540-61142-8_588)
- 229 Raviart, P.-A., & Thomas, J.-M. (1977). A mixed finite element method for 2nd order elliptic
230 problems. In I. Galligani & E. Magenes (Eds.), *Mathematical aspects of finite element*
231 *methods* (Vol. 606, pp. 292–315). <https://doi.org/10.1007/BFb0064470>
- 232 Schöberl, J. (2014). *C++ 11 implementation of finite elements in NGSolve* (No. 30/2014).
233 Vienna University of Technology. <https://doi.org/10.500.12708/28346>
- 234 Scroggs, M. W. (2021). Symfem: A symbolic finite element definition library. *Journal of Open*
235 *Source Software*, 6(64), 3556. <https://doi.org/10.21105/joss.03556>
- 236 Scroggs, M. W., Baratta, I. A., Richardson, C. N., & Wells, G. N. (2022). Basix: A runtime
237 finite element basis evaluation library. *Journal of Open Source Software*, 7(73), 3982.
238 <https://doi.org/10.21105/joss.03982>
- 239 Scroggs, M. W., Brubeck, P. D., Dean, J. P., Dokken, J. S., & Marsden, I. (2026). *DefElement:*
240 *An encyclopedia of finite element definitions*. To appear in Computational Science and
241 Engineering. <https://doi.org/10.1007/s44207-026-00011-0>
- 242 Scroggs, M. W., Dokken, J. S., Richardson, C. N., & Wells, G. N. (2022). Construction
243 of arbitrary order finite element degree-of-freedom maps on polygonal and polyhedral
244 cell meshes. *ACM Transactions on Mathematical Software*, 48(2), 18:1–18:23. <https://doi.org/10.1145/3524456>
- 245 Scroggs, M. W., & Fierro-Piccardo, I. ({2025--2026}). *ndelement-dual*. [https://github.com/](https://github.com/mscroggs/ndelement-dual)
246 [mscroggs/ndelement-dual](https://github.com/mscroggs/ndelement-dual).

- 248 Scroggs, M. W., & Wells, G. N. (2026). *Algorithm: Computation of finite element degree-of-*
249 *freedom transformation matrices*. in preparation.
- 250 Śmigaj, W., Betcke, T., Arridge, S., Phillips, J., & Schweiger, M. (2015). Solving boundary
251 integral problems with BEM++. *ACM Transactions on Mathematical Software*, 41(2),
252 6:1–6:40. <https://doi.org/%7B10.1145/2590830%7D>
- 253 Steinbach, O. (2008). *Numerical approximation methods for elliptic boundary value problems*.
254 Springer-Verlag. <https://doi.org/10.1007/978-0-387-68805-3>
- 255 van 't Wout, E., G  lat, P., Betcke, T., & Arridge, S. (2015). A fast boundary element
256 method for the scattering analysis of high-intensity focused ultrasound. *The Journal of the*
257 *Acoustical Society of America*, 138(5), 2726–2737. <https://doi.org/10.1121/1.4932166>
- 258 Watson, F. M., Andre, D., & Lionheart, W. R. B. (2024). Resolving full-wave through-wall
259 transmission effects in multi-static synthetic aperture radar. *Inverse Problems*, 40(8),
260 085009. <https://doi.org/10.1088/1361-6420/ad5b83>

DRAFT